Solution Architecture & Project Approach: Distributor Touchpoint & Network Expansion Platform

1. Executive Summary

The objective of this project is to build a unified **Web-based Geographic Information System (GIS) and Analytics Platform** for the Parts & Accessories department. This platform will visualize touchpoints, demarcate distributor territories, predict market potential, and digitally manage network expansion. To meet the specific requirements of the Business Requirements Document (BRD), a custom Web Application powered by a spatial database and a Machine Learning (ML) engine is the recommended approach.

(Note: Per recent scope updates, this solution focuses exclusively on a Web Application, removing the Mobile app dependency).

2. Expected Data Requirements from Business

Before development begins, the Analytics & Data Engineering teams require the following datasets:

1. **Geocoded Master Data (Lat/Long):**
    - Internal locations: Mother Warehouses (MW), Additional Warehouses (AW), Retail Offices (RO), and Dealer touchpoints.
    - Customer locations: Independent Workshops, Traders, MASS (Manufacturer Authorized Service Stations), and fleet owners.
2. **Historical Performance Data:** Parts sales volume and revenue (last 3-5 years) mapped to current touchpoints.
3. **Market / Vehicle Parc Data:** Number of company vehicles on the road, mapped at a pin-code, district, or grid level.
4. **Current Territory Logic:** Existing business rules defining distributor boundaries to seed the initial digital maps.

3. Step-by-Step Solution Approach

Step 1: Spatial Data Foundation (Database)

- **Technology:** PostgreSQL with the PostGIS extension.
- **Action:** Ingest all geographical data into the spatial database.
- **BRD Alignment:** PostGIS allows the system to run complex geographic queries (e.g., checking if a proposed touchpoint falls within a restricted polygon), forming the backbone for strict territory demarcation and dispute avoidance.

Step 2: Machine Learning & Analytics Engine (The "Brain")

- **Technology:** Python (Scikit-Learn, XGBoost, GeoPandas).
- **Action 1 (Market Potential):** Train predictive models using Vehicle Parc, historical sales, and regional economic data to calculate the potential parts revenue (₹) for 5x5 km geographic grids across India.
- **Action 2 (White-Space Analysis):** Use spatial clustering algorithms (like DBSCAN) on Independent Workshop locations. Subtract existing Distributor coverage areas to identify and flag dense **"Untapped Potential Pockets."**

Step 3: Interactive Web Portal Development (Frontend UI)

- **Technology:** React.js integrated with Mapbox GL JS or Google Maps Platform.
- **Action:** Build a responsive web application capable of rendering millions of data points on an interactive India map without latency. Include polygon drawing tools for territory management.

Step 4: Digital Touchpoint Workflow & Demarcation

- **Action:** Develop a workflow engine (Node.js or Python FastAPI) that connects the map to business logic.
- **BRD Alignment:** Admins can physically draw and lock distributor territories. If a distributor requests a new Retail Office (RO), the system validates the coordinates against territory boundaries to ensure zero cannibalization and zero disputes.

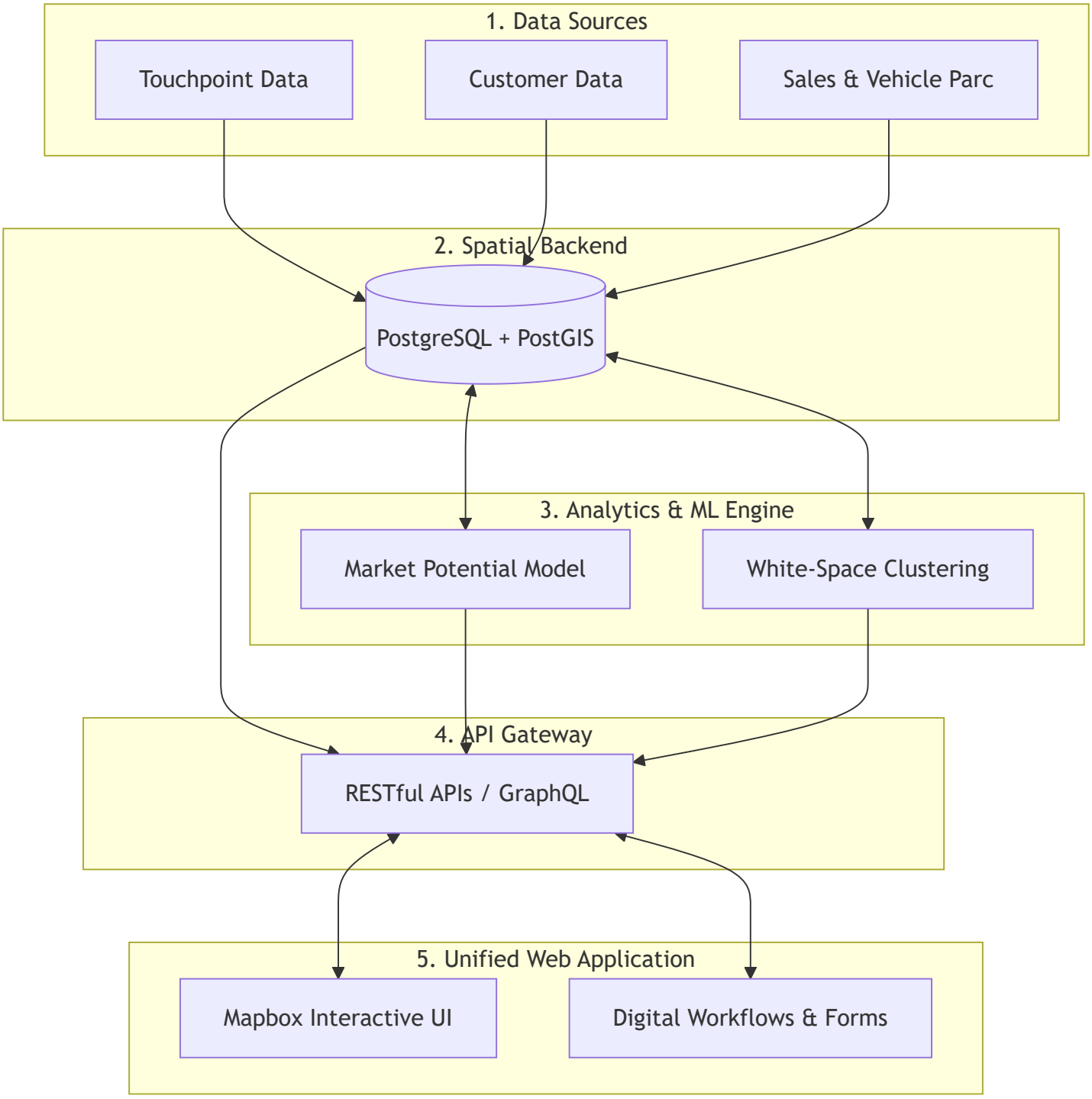
4. Role-Based Access Control (RBAC) Matrix

Role	Access Level	Capabilities
Organization Admin	Pan-India	View all regions; Draw/Lock territory boundaries; View all ML insights; Approve/Reject new RO requests.
Distributor User	Restricted (Own Territory)	View own territory boundaries; View assigned customers; See ML-generated "Untapped Pockets" within their territory; Submit requests for new ROs.

5. System Diagrams

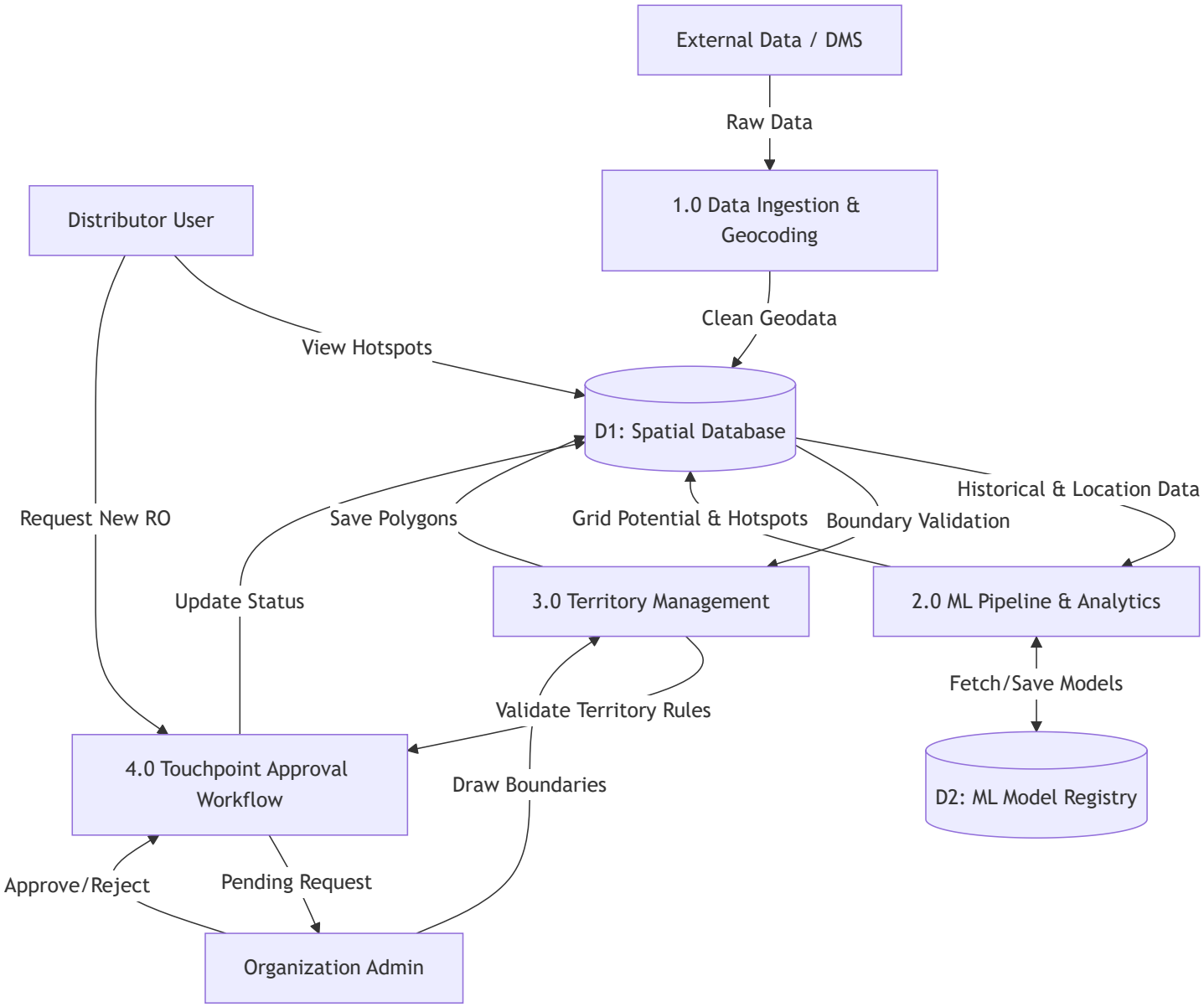
5.1 Solution Approach Diagram

This diagram outlines the high-level conceptual components of the platform.



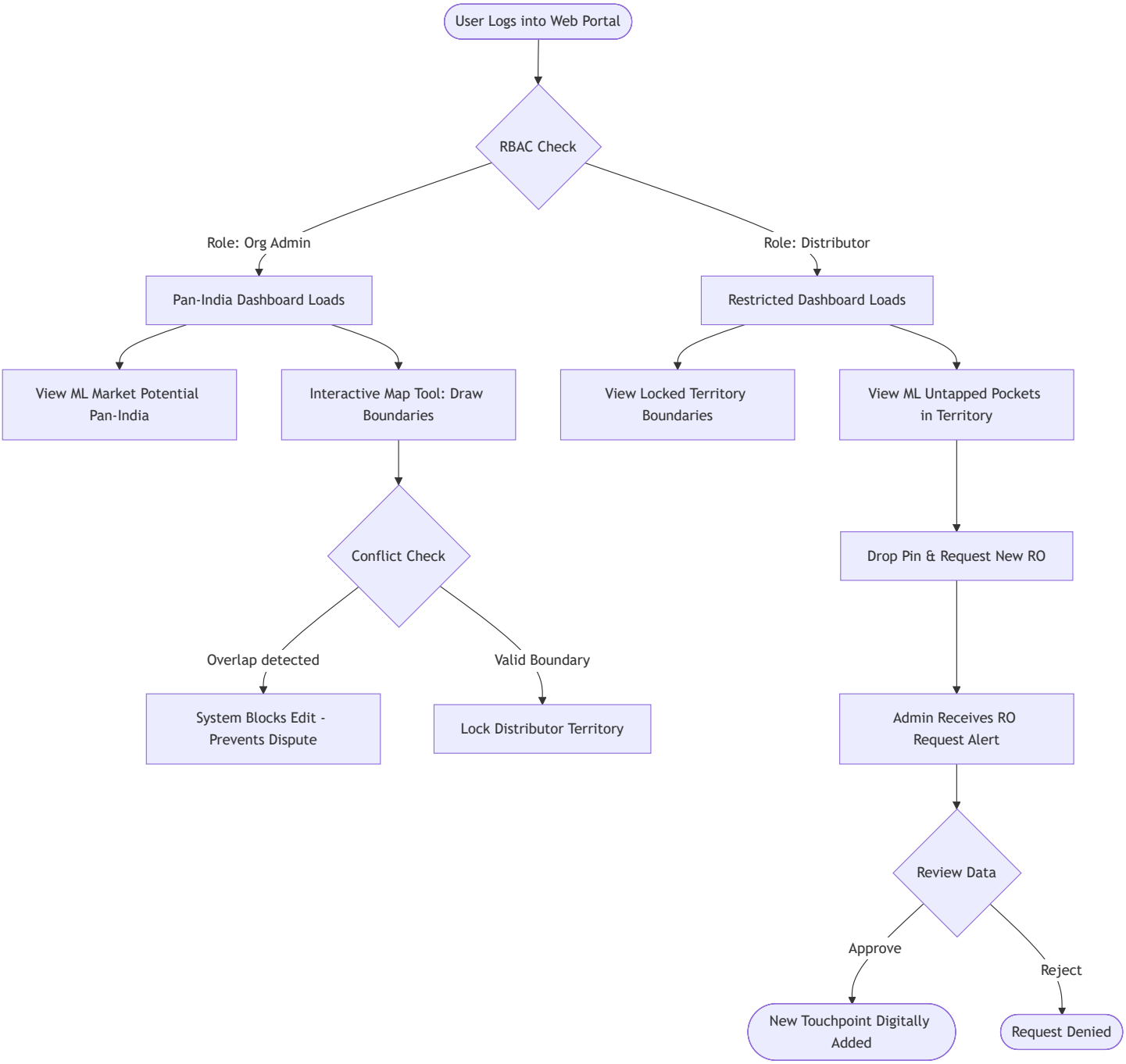
5.2 Data Flow Diagram (DFD Level 1)

This diagram maps how data moves through the system between entities, processes, and storage.



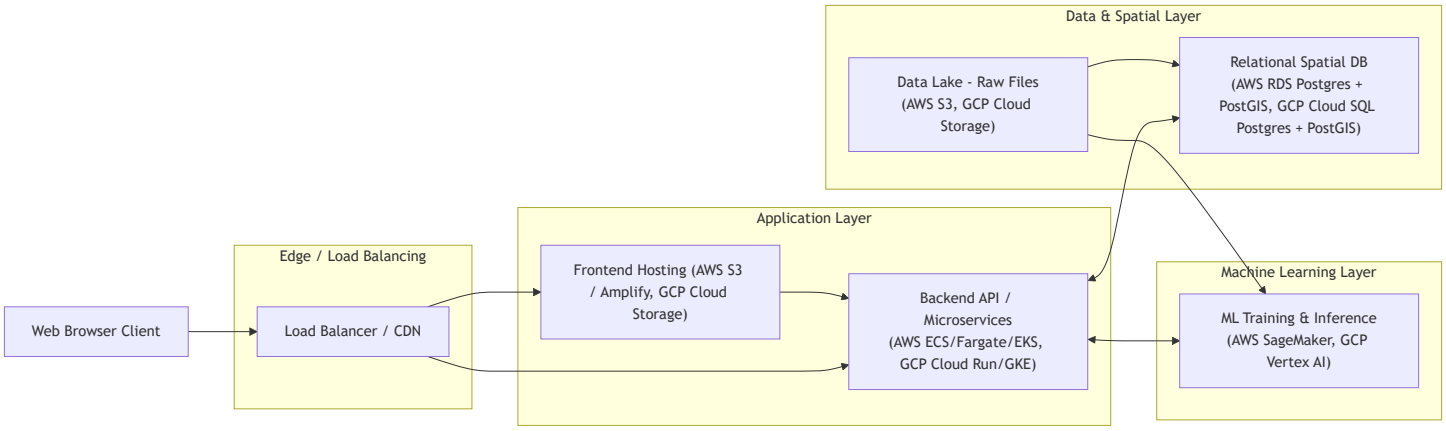
5.3 User to End-Result Flow (with RBAC)

This diagram shows the distinct user journeys for the Admin vs. the Distributor.



5.4 Proposed Architecture Diagram (AWS & GCP equivalents)

This diagram maps the application layers to specific cloud-native services for deployment on either Amazon Web Services (AWS) or Google Cloud Platform (GCP).



6. Recommended Technology Stack Summary

Frontend Application: React.js, Tailwind CSS (for UI).

Mapping Engine: Mapbox GL JS (Optimized for massive spatial datasets).

Backend API: Node.js (Express) OR Python (FastAPI).

Database: PostgreSQL configured with the PostGIS extension.

Machine Learning / Analytics: Python (Pandas, GeoPandas, Scikit-learn, XGBoost).

Cloud Hosting: Highly available deployment on AWS or GCP utilizing managed databases and containerized application hosting.